

UNIDAD 1

FUNDAMENTOS DE PROGRAMACIÓN

PROGRAMACIÓN
CFGS DAW

Adaptado por: José Miguel Blázquez
2023/2024

Licencia



[CC BY-NC-SA 3.0 ES](#) Reconocimiento - No Comercial - Compartir Igual (by-nc-sa)

No se permite un uso comercial de la obra original ni de las posibles obras derivadas, la distribución de las cuales se debe hacer con una licencia igual a la que regula la obra original. NOTA: Esta es una obra adaptada por José Miguel Blázquez, cuyos autores originales son Carlos Cacho, Raquel Torres, Lionel Tarazon y Fco. Javier Valero.

Nomenclatura

A lo largo de este tema se utilizarán distintos símbolos para distinguir elementos importantes dentro del contenido. Estos símbolos son:



Importante



Atención



Interesante

ÍNDICE DE CONTENIDOS

1.INTRODUCCIÓN	4
2.ALGORITMOS Y PROGRAMAS.....	4
3.PARADIGMAS Y LENGUAJES DE PROGRAMACIÓN	5
3.1 Paradigmas.....	5
3.2 Lenguajes de programación.....	6
3.3 Procesadores de lenguajes	9
3.4 Software de programación	9
4.TIPOS DE APLICACIONES	11
5.CICLO DE VIDA DE UN PROGRAMA	11
5.1 Fase de análisis	11
5.2 Fase de diseño	12
5.3 Fase de implementación (desarrollo o codificación)	12
5.4 Fase de pruebas	13
5.5 Fase de despliegue y mantenimiento.....	13
6.DOCUMENTACIÓN	13
7.ELEMENTOS DE UN PROGRAMA	14
7.1 Constantes	14
7.2 Variables	14
7.3 Expresiones	14
7.4 Operadores.....	15
7.4.1 Relacionales	15
7.4.2 Aritméticos.....	15
7.4.3 Lógicos o booleanos.....	16
7.4.4 Paréntesis ()	17
7.4.5 Orden de evaluación de los operadores	18
8.ERRORES EN EL DESARROLLO DE PROGRAMAS.....	19
9.REPRESENTACIÓN.....	19
8.1 Diagramas de flujo (ordinogramas)	19
8.2 Pseudocódigo	22

UD01. FUNDAMENTOS DE PROGRAMACIÓN

1. INTRODUCCIÓN

La razón principal por la que una persona utiliza un ordenador es para resolver problemas (en el sentido más general de la palabra), o, en otras palabras, procesar una información para obtener un resultado a partir de unos datos de entrada.

Los ordenadores resuelven los problemas mediante la utilización de programas escritos por los programadores. Los programas de ordenador no son entonces más que métodos para resolver problemas. Por ello, para escribir un programa, lo primero es que el programador sepa resolver el problema que estamos tratando (o tenga la ayuda especializada pertinente, no podemos saber de todo...).

El programador debe identificar cuáles son los datos de entrada y a partir de ellos obtener los datos de salida, es decir, la solución, a la que se llegará por medio del procesamiento de la información que se realizará mediante la utilización de un método para resolver el problema que denominaremos algoritmo.

Cada producto software es (o puede ser) diferente al resto, porque está desarrollado para un cliente diferente, o para cumplir un propósito diferente. Entonces, desarrollar este producto implica unos pasos previos: entender lo que tenemos que hacer, hacer un diseño previo, e implementarlo, como veremos más adelante. Por lo tanto, no podemos comparar el desarrollo de software con la producción industrial (como la fabricación de teclados, por ejemplo), donde todo está mucho más automatizado. El desarrollo de software requiere la creación de un proyecto de software y un grupo de personas trabajando de manera coordinada. Además, el software no se rompe con el tiempo, aunque su rendimiento puede verse reducido por sus propias actualizaciones y mejoras.

2. ALGORITMOS Y PROGRAMAS

Por algoritmo entendemos una **secuencia ordenada de pasos, descrita sin ambigüedades, que conducen a la solución de un problema dado**. que además cumplen las siguientes características:

- Tiene un número finito de pasos
- Acaba en un tiempo finito. Si no acabase nunca, no se resolvería el problema.
- Todas las operaciones deben estar definidas de forma precisa y sin ambigüedad.
- Puede tener varios datos de entrada y como mínimo un dato de salida.

Un claro ejemplo de algoritmo es una receta de cocina, donde tenemos unos pasos que hay que seguir en un orden y deben de estar bien definidos, tiene un tiempo finito y tiene unos datos de entrada

(ingredientes) y una salida (el plato).

Por ejemplo, el algoritmo para **freír un huevo** podría ser el siguiente:

Datos de entrada: Huevo, aceite, sartén, fuego.

Datos de salida: huevo frito.

Procedimiento:

1. Poner el aceite en la sartén.
2. Poner la sartén al fuego.
3. Cuando el aceite esté caliente, cascar el huevo e introducirlo.
4. Cubrir el huevo de aceite.
5. Cuando el huevo esté hecho, retirarlo.

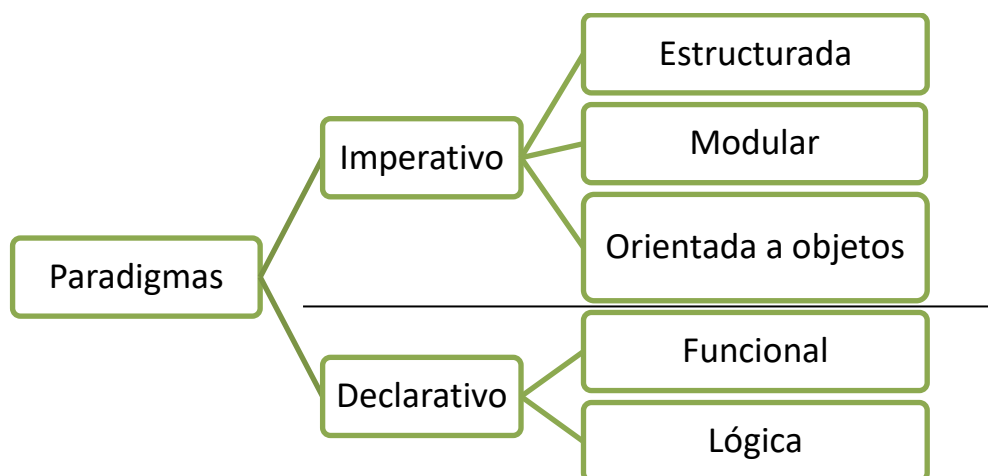
La codificación de un algoritmo en un ordenador se denomina **programa**, es decir, la traducción a las instrucciones en un lenguaje de programación.

3. PARADIGMAS Y LENGUAJES DE PROGRAMACIÓN

Existen muchas formas diferentes de hacer aplicaciones, así que deberemos seguir un paradigma que establezca un patrón en la creación de nuestros programas. Un paradigma de programación es un modelo básico para el diseño y la implementación de programas. Este modelo determinará cómo será el proceso de diseño y la estructura final del programa. El paradigma representa un **enfoque** particular o **filosofía** para la **construcción** de **software**. Cada uno tendrá sus ventajas e inconvenientes, será más o menos apropiado, pero no es correcto decir que exista uno mejor que los demás.

3.1 Paradigmas

En el siguiente esquema puedes ver los diferentes paradigmas de programación, y los estilos de programación asociados:



3.2 Lenguajes de programación

Un lenguaje de programación es un lenguaje formal (es decir, un lenguaje con reglas gramaticales bien definidas) que le proporciona a una persona, en este caso el programador, la capacidad de escribir (o programar) una serie de instrucciones o secuencias de órdenes en forma de algoritmos con el fin de controlar el comportamiento físico o lógico de un sistema informático, de manera que se puedan obtener diversas clases de datos o ejecutar determinadas tareas. A todo este conjunto de órdenes escritas mediante un lenguaje de programación se le denomina programa.

Los lenguajes pertenecen a uno (puros) o varios (híbridos) paradigmas de programación.

```
public class ProgramaSuma
{
    public static void main(String args[])
    {
        System.out.println("Hola, mundo! ");
    }
}
```

Arriba hay un ejemplo de código escrito en lenguaje Java. Un lenguaje basado en el paradigma orientado a objetos con soporte a otros (**híbrido** o multiparadigma), ya que combina características de otros paradigmas como el funcional. Un lenguaje de programación es el conjunto de reglas sintácticas y semánticas, símbolos y palabras especiales establecidas para la construcción de programas. Veamos sus componentes:

- **Gramática del lenguaje:** reglas aplicables al conjunto de símbolos y palabras especiales del lenguaje de programación para la construcción de sentencias correctas.
- **Léxico:** es el conjunto finito de símbolos y palabras especiales, es el vocabulario del lenguaje.
- **Sintaxis:** son las posibles combinaciones de los símbolos y palabras especiales. Está relacionada con la forma de los programas. Cada lenguaje tiene sus reglas sintácticas propias y particulares.
- **Semántica:** es el significado de cada construcción del lenguaje, la acción que se llevará a cabo.

El lenguaje Java es el lenguaje de programación que vamos a aprender en este curso, debido a varios motivos, entre ellos su potencia, multiplataformidad y capacidad de uso en multitud de circunstancias.

Los lenguajes de programación pueden ser clasificados en función de lo cerca que estén del lenguaje humano o del lenguaje de los computadores. El lenguaje de los computadores son códigos binarios, es decir, secuencias de unos y ceros. Detallaremos seguidamente las características principales de los lenguajes de programación:

A. **Lenguajes de nivel bajo** (cerca de la arquitectura de la máquina): dependencia clara del hardware.

- **Lenguaje máquina**

El más primitivo de los lenguajes. Es una colección de dígitos binarios o bits (0 y 1) que la computadora lee e interpreta y es el único idioma que las computadoras entienden. Por ejemplo, la instrucción “*sumar los números 9 y 11*” podría ser:

0001 1100 0111 0011 1001 1011

- **Lenguaje ensamblador**

El primer intento de sustitución del lenguaje máquina por uno más cercano al utilizado por los humanos. Se almacena como texto (tal como programas de alto nivel) y consiste en una serie de instrucciones que corresponden al flujo de órdenes ejecutables por un microprocesador.

Sin embargo, el microprocesador no comprende el lenguaje ensamblador, por lo que se debe convertir a lenguaje máquina mediante un programa llamado “Ensamblador”. Con este lenguaje el programador tiene el control total de la máquina. Las instrucciones son *mnemotécnicas*, como por ejemplo ADD, DIV, MOV, etc. Una instrucción del ejemplo anterior sería **ADD 9,11**.

B. **Lenguajes de nivel alto** (más cerca del humano): abstracción respecto a la arquitectura, más fáciles de entender y manipular.

- **Lenguajes interpretados** (Python, JavaScript....)

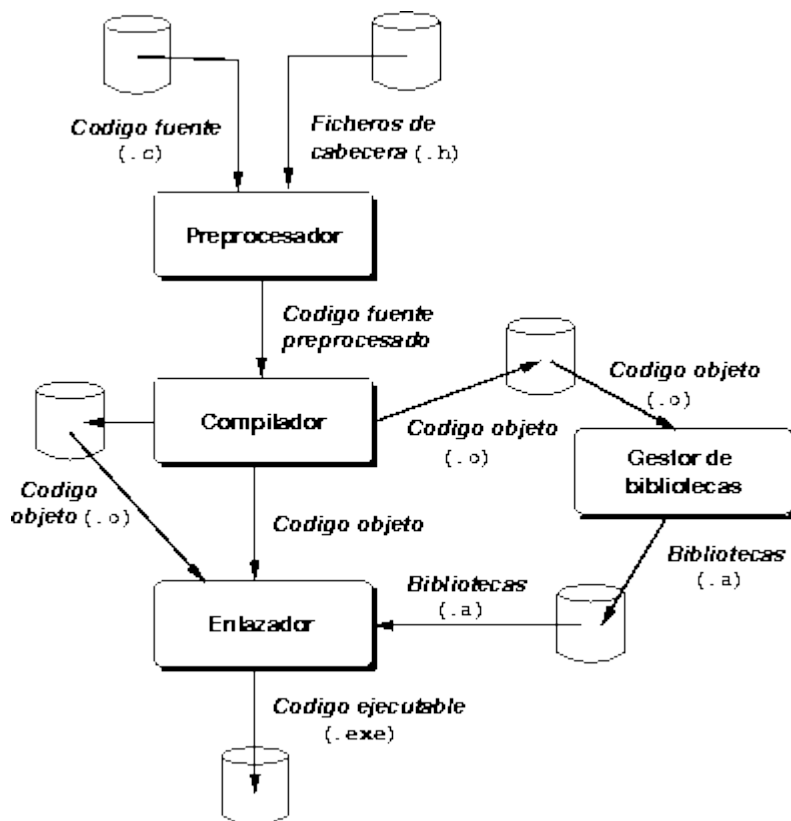
Su ejecución se realiza a través de un intérprete. Un intérprete es un programa traductor de un lenguaje de alto nivel en el que el proceso de traducción y de ejecución se llevan a cabo simultáneamente, es decir, la instrucción se pasa a lenguaje máquina y se ejecuta directamente. No se genera programa objeto, ni programa ejecutable.

Características de los lenguajes interpretados:

- Programas de menor tamaño que los generados por un compilador, al no guardar el programa traducido a código máquina.
- Algo más lentos, ya que han de ser traducidos durante su ejecución.
- Necesitan disponer en la máquina del programa intérprete ejecutándose, algo que no es necesario en el caso de un programa compilado, para los que sólo es necesario tener el programa ejecutable para poder utilizarlo.

- **Lenguajes compilados** (C, C++, Pascal...)

El compilador realizará la traducción del código fuente y además informará de los posibles errores. Una vez subsanados, se generará el programa traducido a código máquina, conocido como **código objeto**. Pero este programa aún no podrá ser ejecutado hasta que no se le añadan los módulos de enlace o bibliotecas, durante el **proceso de enlazado**. Una vez finalizado el enlazado, se obtiene el **código ejecutable**. Veamos un ejemplo del proceso de compilación para un programa escrito en lenguaje C y ejecución en entorno Windows.



- **Lenguajes pseudo-compilados o pseudo-interpretados (C#, Java...)**

Una combinación de los 2 anteriores, como, por ejemplo: **Java**. Su código fuente se compila para obtener el código binario en forma de **bytecodes**, estructuras parecidas a las instrucciones máquina, con la importante propiedad de no ser dependientes de ningún tipo de máquina. La Máquina Virtual Java se encargará de interpretar este código y, para su ejecución, lo traducirá en ese momento al código máquina del procesador en particular sobre el que se esté trabajando.

3.3 Procesadores de lenguajes

Las computadoras no pueden entender ninguno de los lenguajes de programación que los humanos usan para crear sus programas. Para que funcionen, sus instrucciones deben traducirse a un idioma que las computadoras puedan entender. Este lenguaje se llama código máquina y está compuesto por bits (ceros y unos).

Si queremos traducir un determinado lenguaje de programación a código máquina, utilizamos una herramienta llamada compilador, aunque esta afirmación no es del todo cierta debido a la complejidad y casuística actual de lenguajes de programación. Hay algunos procesadores de lenguajes diferentes que se pueden usar dependiendo del lenguaje en sí:

- **Compiladores:** traducen el código escrito en un lenguaje específico a código máquina, y generan un archivo ejecutable con el resultado. Por ejemplo, si compilamos un programa escrito en C bajo Windows, obtendremos un archivo EXE que podremos ejecutar.
- **Intérpretes:** traducen del lenguaje de programación especificado al código de máquina “sobre la marcha”. En otras palabras, no generan ningún archivo ejecutable. Entonces, cada vez que necesitamos ejecutar el programa, necesitamos tener disponible el archivo fuente. Este tipo de procesador de lenguaje es muy habitual en lenguajes como PHP o Python. De esta forma, el tiempo de respuesta aumenta un poco, pero el programa se puede ejecutar en múltiples plataformas.
- **Máquinas virtuales:** una solución intermedia entre compiladores e intérpretes es la que utilizan lenguajes como Java. Estos programas no se compilan en un código de máquina nativo (no hay un archivo EXE en Java, por ejemplo), y tampoco se interpretan. Java compila el código fuente y lo traduce a su propio código de máquina intermedio. Luego, ejecuta su máquina virtual (JVM, Java Virtual Machine), que se encarga de interpretar y ejecutar este código cada vez que lo necesitemos. De esta forma, no necesitamos tener disponible el código fuente antes de ejecutar el programa, y tampoco dependemos de una plataforma determinada (Windows, Linux, etc.). Solo necesitamos tener una JVM instalada en nuestro sistema para poder ejecutar nuestros programas Java. Lo mismo ocurre con C# y su plataforma virtual .NET .

3.4 Software de programación

Por software de programación entendemos el conjunto de todas las herramientas que le permiten al programador: crear, editar códigos, depurar, mantener y desplegar los proyectos.

Algunos de los distintos programas por los que pasará el proyecto para gestionarlo son:

- **Editores de código o texto:** necesarios para escribir el código fuente. Suelen incorporar funciones de ayuda, como el auto-completado, indentación, marcado de errores sintácticos y la refactorización.
- **Compiladores:** traducen el código ingresado a lenguaje de máquina generando un código

binario ejecutable en CPU y dependiente de la arquitectura.

- **Depuradores:** sirven para optimizar el tiempo de desarrollo mediante el monitoreo de la ejecución de un programa, el seguimiento a los valores de ciertas variables, las referencias a objetos en memoria y, por ende, nos ayuda a corregir errores.
- **Interpretores o traductores:** el traductor (o intérprete) carga el código ingresado y traduce las instrucciones para que el programa pueda ser ejecutado en el momento.
- **IDE:** El IDE (Integrated Development Environment) o Entorno de Desarrollo Integrado, es una aplicación informática que proporciona una serie de servicios que facilitan la programación de software, tales como: funciones de autocompletado; un editor de código fuente; gestión de conexiones a bases de datos; integración con sistemas de control de versiones; simuladores de dispositivos; un depurador para agilizar el proceso de desarrollo de software, entre otros.

Veamos el equivalente al ejemplo anterior (Java) en otros lenguajes populares:

C++

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hola, mundo!" << endl;
    return 0;
}
```

C#

```
using System;

class Program {
    static void Main() {
        Console.WriteLine("Hola, mundo!");
    }
}
```

Python

```
print("Hola, mundo!")
```

PHP

```
<?php
    echo "Hola mundo";
?>
```

4. TIPOS DE APLICACIONES

En **general**, podemos clasificar los programas en 4 formatos fundamentales:

- **Aplicaciones de consola:** programas que se ejecutan en la línea de comandos o terminal. Estas aplicaciones son conocidas por su simplicidad y eficiencia. En lugar de una interfaz gráfica compleja las aplicaciones de consola se basan en texto y comandos para interactuar con el usuario. Lógicamente tienen sus limitaciones.
- **Aplicaciones con interfaz gráfica de usuario (GUI):** utilizan elementos visuales como ventanas, botones y menús, para facilitar la interacción del usuario. Estas aplicaciones son conocidas por su “facilidad” de uso y su capacidad para brindar una experiencia visualmente atractiva. Se requieren librerías especializadas para poder desarrollarlas, como Tkinter para Python, WinForms o WPF para C#, Swing o JavaFX para Java, etc...
- **Servicios (Windows) o *daemons* (Linux):** programas que se ejecutan en segundo plano y están diseñados para realizar tareas específicas en cooperación con el Sistema Operativo. A diferencia de las anteriores, los servicios no interactúan directamente con los usuarios. En su lugar, ofrecen funcionalidades y recursos que otros programas pueden usar.
- **Aplicaciones web:** programas que se ejecutan en un servidor y se acceden a través de un navegador web. Estas aplicaciones han ganado popularidad debido a su capacidad para funcionar en diferentes plataformas y sistemas operativos sin necesidad de instalar software adicional, sólo con un navegador, en lo que conocemos como arquitectura en la nube (cloud).

Normalmente, cuando se aprende a programar se empieza por aplicaciones en consola, que son más sencillas, y luego se va realizando la transición hacia otros formatos más complejos.

5. CICLO DE VIDA DE UN PROGRAMA

La creación de cualquier programa (software o sw) implica la realización de muchas fases que vamos a resumir en **5 pasos**:

1. Análisis.
2. Diseño.
3. Implementación (desarrollo).
4. Pruebas.
5. Mantenimiento.

5.1 Fase de análisis

En esta fase se recopilan y analizan los **requisitos** del software a desarrollar. Esto implica comprender las necesidades de los usuarios, los objetivos del software y las restricciones técnicas y comerciales. El resultado es la definición clara de lo que debe hacer el software.

Esta fase implica hablar con las personas que quieren el software. Puede ser un cliente, un usuario final o alguien que tenga una necesidad específica. El analista debe hacer preguntas para entender lo que quieren que haga el software y cómo quieren que lo haga.

La idea es recopilar todos estos detalles y escribirlos de manera clara para que tanto el equipo de desarrollo como el cliente o usuario comprendan exactamente qué esperar. Esto ayuda a evitar malentendidos más adelante cuando se desarrolle el software. Es como tener una receta detallada antes de hornear el pastel, solo que, en este caso, estamos creando instrucciones para el software a construir.

Es como sentarse y hablar con alguien que te pide que le hagas un pastel. Imagina que eres un chef de pastelería y alguien te encarga un pastel para una boda. En esta primera fase del desarrollo de software, tu objetivo es entender exactamente qué tipo de pastel quieren. Aquí es donde hablas con la persona que quiere el pastel y haces preguntas como: "¿De qué sabor quieres el pastel?", "¿Necesitas que tenga algún relleno especial?", "¿Cuántas personas deben comer?", y otras preguntas para obtener todos los detalles necesarios.

5.2 Fase de diseño

En esta fase se crea un diseño detallado del software en base a los requisitos y especificaciones recopilados anteriormente. Esto incluye diseñar la arquitectura general del sistema, la interfaz de usuario, la estructura de datos y los componentes del software. El diseño se centra en la planificación de cómo se implementarán los requisitos.

Es como crear un plano detallado antes de construir el software. Aquí es donde se parte de la información obtenida en la fase de Análisis y se empieza a diseñar cómo se verá y cómo funcionará el software, y a través de qué tecnologías concretas (lenguajes de programación, bases de datos....).

Imagina que trabajas en un equipo que está haciendo una aplicación web para pedidos de comida online. En esta fase se decidirá cómo se verá la pantalla de inicio, dónde estarán los botones para elegir la comida, cómo se mostrarán los menús, cómo funcionará el proceso de pago, qué servidores soportarán la aplicación, qué tipo de base de datos concreta almacenará la información, etc... Esto implica tomar decisiones sobre cómo organizar todas las partes del software, cómo se comunicarán entre sí y cómo se verá la interfaz que los usuarios usarán. Es como dibujar un mapa detallado de instrucciones para el equipo de desarrollo, que entrará en funcionamiento en la siguiente fase.

5.3 Fase de implementación (desarrollo o codificación)

En esta etapa, se parte de lo obtenido en la fase de diseño (diagramas, tablas, esquemas...) que facilitan la delineación de las tareas o pasos que seguirá el programa. En esta fase **se convierte el algoritmo en programa**, escribiéndolo en un lenguaje de programación de alto nivel como Java, C#, PHP, etc. La implementación del programa suele ser una tarea pesada que requiere un conocimiento completo de las características del lenguaje elegido para conseguir un programa eficaz. Sin embargo, si el diseño del algoritmo se ha realizado en detalle de forma correcta y con buena legibilidad, el proceso de implementación funcionará mucho mejor.

En esta fase el equipo de programación escribe el código del software basado en los diseños creados. Aquí es donde se traducen los diseños en código ejecutable utilizando lenguajes de programación y herramientas específicas. Esta es la fase donde se construye el núcleo funcional del software.

Siguiendo el ejemplo del pastel. Imagina que ya tienes la receta del pastel y has dibujado cómo se verá y cómo se organizarán los ingredientes. Ahora es el momento de mezclar todos esos ingredientes (elegidos en la fase de diseño), ponerlos en el horno y esperar a que el pastel se cocine.

5.4 Fase de pruebas

Una vez que se ha completado la implementación, se realizan pruebas exhaustivas para asegurarse de que el software cumpla con los requisitos y funcione como se espera. Esto incluye pruebas de funcionalidad, pruebas de rendimiento, pruebas de seguridad y otros tipos de pruebas. Los errores se identifican, se corrigen y se verifica que el software funcione correctamente antes de entregarlo al solicitante. Imagina que has horneado el pastel y ya está listo (teóricamente). Pero antes de entregarlo, obtienes una pequeña porción para probarlo y asegurarte de que tiene el sabor correcto y la textura adecuada. Si algo no está bien, podrías ajustar la receta o hacer cambios para que el pastel sea perfecto.

5.5 Fase de despliegue y mantenimiento

En esta etapa el software se pone a disposición de los usuarios finales. Puede implicar la instalación del software en servidores o dispositivos específicos y la realización de actividades para asegurarse de que el software esté listo para su uso en el entorno de producción. También se puede proporcionar formación y soporte a los usuarios.

Dentro de esta fase, y aunque suele ser algo más a medio o largo plazo, suele estar también el mantenimiento. Después de que el software haya sido utilizado por un tiempo, es posible que surjan problemas o que los usuarios tengan comentarios sobre cómo mejorarlo, o bien necesidades de ampliación de funcionalidades y actualizaciones. Esto puede incluir la corrección de errores, la adición de nuevas características o la optimización del rendimiento.

6. DOCUMENTACIÓN

La mayor parte de los proyectos exigen la realización de una **planificación** previa. Esta planificación debe determinar el modelo de ciclo de vida a seguir, los plazos para completar cada fase y los recursos necesarios en cada momento. Todo esto se debe plasmar en una documentación completa y detallada de toda la aplicación.

La **documentación** asociada al software puede clasificarse en interna y externa. La **interna** corresponde a la que se incluye **dentro del código fuente** de los programas, la cual nos aclara aspectos de las propias instrucciones del programa. La **externa** es la que corresponde a todos los documentos relativos al diseño de la aplicación, a la descripción de la misma y sus módulos correspondientes, a los manuales de usuario y los manuales de mantenimiento.

7. ELEMENTOS DE UN PROGRAMA

Entendemos por **elemento** de un programa **todo aquello que pueda ser manipulado por las instrucciones**. En ellos se almacenarán tanto los datos de entrada como los de salida (resultados).

Sus atributos son:

- **Nombre**: el identificador del objeto.
- **Tipo**: conjunto de valores que puede tomar.
- **Valor**: elemento del tipo que se le asigna.

7.1 Constantes

Son elementos cuyo **valor permanece invariable** a lo largo de la ejecución de un programa. Una constante es la denominación de un valor concreto, de tal forma que se utiliza su nombre cada vez que se necesita referenciarlo (es habitual cuando hablamos de constantes matemáticas o físicas, por ejemplo). Hay algunas bien conocidas, como el número PI, el número e, etcétera...

También son utilizadas las constantes para facilitar la modificabilidad de los programas, es decir para hacer más independientes ciertos datos del programa. Por ejemplo, supongamos un programa en el que cada vez que se calcula un importe al que se debe sumar el IVA escribiéramos directamente el valor 0.21. Si en el futuro tuviéramos que cambiar este índice tendríamos que ir buscando a lo largo del programa y modificando dicho valor cada vez que aparezca; mientras que, si le damos nombre y le asignamos un valor, podremos modificar dicho valor con mucha más facilidad.

7.2 Variables

Son elementos cuyo **valor puede ser modificado** a lo largo de la ejecución de un programa, y permiten almacenarse en memoria para ser usados. Por **ejemplo**: una variable para calcular el área de una circunferencia determinada, una variable para calcular el precio de una factura, etc.

7.3 Expresiones

Las expresiones **según el resultado que produzcan** se clasifican en:

- **Numéricas**: Son las que producen resultados de tipo numérico. Se construyen mediante los operadores aritméticos.

Por ejemplo:

```
pi * sqr(x)
(2*x)/3
```

- **Alfanuméricas**: Son las que producen resultados de tipo alfanumérico. Se construyen mediante operadores alfanuméricos.

Por ejemplo:

```
"Don " + "José"
```

- **Booleanas o lógicas:** Son las que producen resultados de tipo Verdadero o Falso. Se construyen mediante los operadores relacionales y lógicos.

Por ejemplo:

```
a < 0  
(a > 1) and (b < 5)
```

7.4 Operadores

Son símbolos que hacen de enlace entre los argumentos o miembros de una expresión.

7.4.1 Relacionales

Se usan para formar expresiones que al ser evaluadas devuelven un valor booleano: verdadero o falso.

Operador	Definición
<	Menor que
>	Mayor que
==	Igual que
>=	Mayor o igual que
<=	Menor o igual que
<>	Distinto que

Ejemplos:

Expresión	Resultado
A' < 'B'	Verdadero, ya que en código ASCII la A está antes que la B
1 < 6	Verdadero
10 < 2	Falso



ASCII (acrónimo inglés de American Standard Code for Information Interchange — Código Estándar Estadounidense para el Intercambio de Información), es un código de caracteres basado en el alfabeto latino, tal como se usa en inglés moderno. Puedes ver la tabla en el siguiente enlace: <http://ascii.cl/es/>

7.4.2 Aritméticos

Se utilizan para realizar operaciones aritméticas.

Operador	Definición
+	Suma
-	Resta
*	Multiplicación
^	Potencia
/	División
%	Resto de la división

Ejemplos:

Expresión	Resultado
$3 + 5 - 2$	6
$24 \% 3$	0
$8 * 3 - 7 / 2$	20.

7.4.3 Lógicos o booleanos

La combinación de expresiones con estos operadores produce el resultado verdadero o falso.

Operador	Definición
No	Negación
Y	Conjunción
O	Disyunción

El comportamiento de un operador lógico se define mediante su correspondiente [tabla de verdad](#), en ella se muestra el resultado que produce la aplicación de un determinado operador a uno o dos valores lógicos. Las operaciones lógicas más usuales son:

- NO lógico (NOT) o negación:

A	NOT A
V	F
F	V

El operador NOT invierte el valor: Si es verdadero (V) devuelve falso (F), y viceversa.

- O lógica (OR) o disyunción:

A	B	A OR B
V	V	V
V	F	V
F	V	V
F	F	F

El operador OR devuelve verdadero (V) si alguno de los dos valores es verdadero. De lo contrario, devuelve Falso (F).

- Y lógica (AND) o conjunción:

A	B	A AND B
V	V	V
V	F	F
F	V	F
F	F	F

El operador AND devuelve Verdadero (V) solo si ambos valores son verdaderos. En cualquier otro caso devuelve Falso (F).

Ejemplos (Suponiendo que $a < b$):

Expresión	Resultado
$9 == (3 * 3)$	Verdadero
$3 <> 2$	Verdadero
$9 = (3*3) \text{ Y } 3 <> 2$	Verdadero
$3 > 2 \text{ Y } b < a$	Verdadero Y Falso = Falso
$3 > 2 \text{ O } b < a$	Verdadero O Falso = Verdadero
$\text{no}(a < b)$	No Verdadero = Falso
$5 > 1 \text{ Y NO}(b < a)$	Verdadero Y no Falso = Verdadero

7.4.4 Paréntesis ()

Anidan expresiones y permiten modificar la precedencia de algunas operaciones.

Ejemplos:

Operación $(3*2) + (6/2)$ Resultado 9

7.4.5 Orden de evaluación de los operadores

A la hora de resolver una expresión, el orden básico a seguir para evaluar los operadores es el siguiente:

1. Paréntesis
2. Potencia ^
3. Multiplicación y división * /
4. Sumas y restas + -
5. Concatenación +
6. Relacionales < <= > >= etc.
7. Negación NOT
8. Conjunción AND
9. Disyunción OR

La evaluación de operadores de igual orden se realiza de izquierda a derecha. Este orden de evaluación tiene algunas modificaciones en determinados lenguajes de programación.

Ejemplo: Evaluación de expresiones aritméticas

Sea $x=3$, evaluar la expresión para obtener el valor de y

$$y = x * x / (2 - x) + (8 - 3 * x) / 12;$$

Sustituir el valor de la variable en la expresión:

$$y = 3 * 3 / (2 - 3) + (8 - 3 * 3) / 12;$$
 Eliminar paréntesis

$$y = 3 * 3 / -1 + (8 - 3 * 3) / 12;$$

$$y = 3 * 3 / -1 + (8 - 9) / 12;$$

Eliminar paréntesis respetando la jerarquía de operadores. El operador * tiene mayor precedencia que el operador -, por lo que se realiza primero $3 * 3$ y después $8 - 9$

$$y = 3 * 3 / -1 - 1 / 12;$$

$$y = 9 / -1 - 1 / 12;$$

$$y = -9 - 1 / 12;$$

$$y = -9 - 0.083;$$

$$y = -9.083;$$
 Evaluar los operadores considerando su jerarquía

8. ERRORES EN EL DESARROLLO DE PROGRAMAS

En el proceso de desarrollo de un programa, así como durante todo el ciclo de vida de una aplicación, se pueden producir distintos errores. Estos pueden ser:

- **Errores sintácticos:** se detectarán en la fase de compilación del código. Suelen venir provocados por problemas en la escritura del código (falta de caracteres, error gramatical).
- **Errores semánticos:** se detectarán una vez compilado el programa, cuando intentemos ejecutarlo. Estos errores impiden que un programa haga lo que estaba previsto correctamente. El código puede compilarse y ejecutarse sin errores, pero el resultado de una operación puede generar un resultado no esperado o inconsistente.

Un ejemplo de esto es la división por cero. Suponga que tiene la instrucción siguiente:

velocidad = kilometros / horas

Si la variable horas recibiera un valor de 0, se produciría un error en tiempo de ejecución en la operación de división. El programa se debe ejecutar para que se pueda detectar este error, pero si horas contiene un valor válido no se producirá el error.

Los errores semánticos son los más difíciles de detectar y corregir. Las herramientas de depuración facilitan el trabajo de encontrarlos y resolverlos.

9. REPRESENTACIÓN

Existen diversas formas de representación de algoritmos. Las más importantes son los diagramas de flujo (también llamados 'ordinogramas') y el pseudocódigo.

8.1 Diagramas de flujo (ordinogramas)

Durante el diseño de un programa y en sus fases de análisis y programación, surge la necesidad de **representar de una manera gráfica los flujos que van a seguir los datos** manipulados por el mismo, así como la secuencia lógica de las operaciones para la resolución del problema.

Esta representación gráfica debe tener las siguientes cualidades:

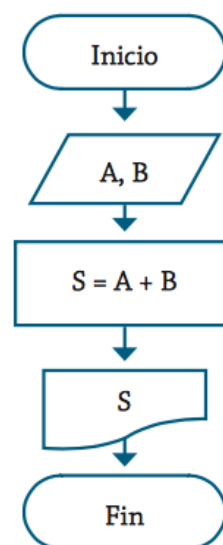
1. Sencillez en su construcción.
2. Claridad en su comprensión.
3. Normalización en su diseño.
4. Flexibilidad en sus modificaciones.

⚡ En la práctica se suelen utilizar indistintamente los términos **diagrama de flujo**, **organigrama** y **ordinograma** para referenciar cualquier representación gráfica de los flujos de datos o de las operaciones de un programa. Es importante diferenciarlos porque no corresponden a las mismas fases de diseño de los programas. Aunque utilicen algunos símbolos comunes, el significado de éstos no es el mismo.

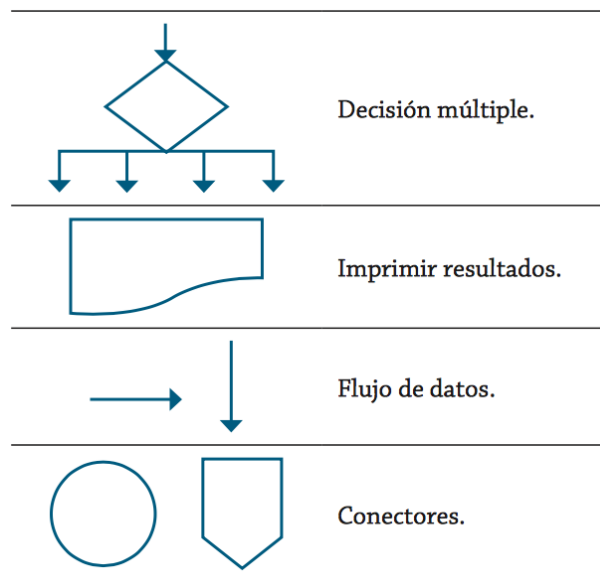
En la representación de ordinogramas es conveniente seguir las siguientes reglas:

- El comienzo del programa figurará en la parte superior del ordinograma.
- El símbolo de comienzo deberá aparecer una sola vez en el ordinograma.
- El flujo de las operaciones será, siempre que sea posible de arriba a abajo y de izquierda a derecha.
- Se evitarán siempre los cruces de líneas utilizando conectores.

En la derecha se muestra un ejemplo muy simple, y, tras él, la simbología estándar a emplear:

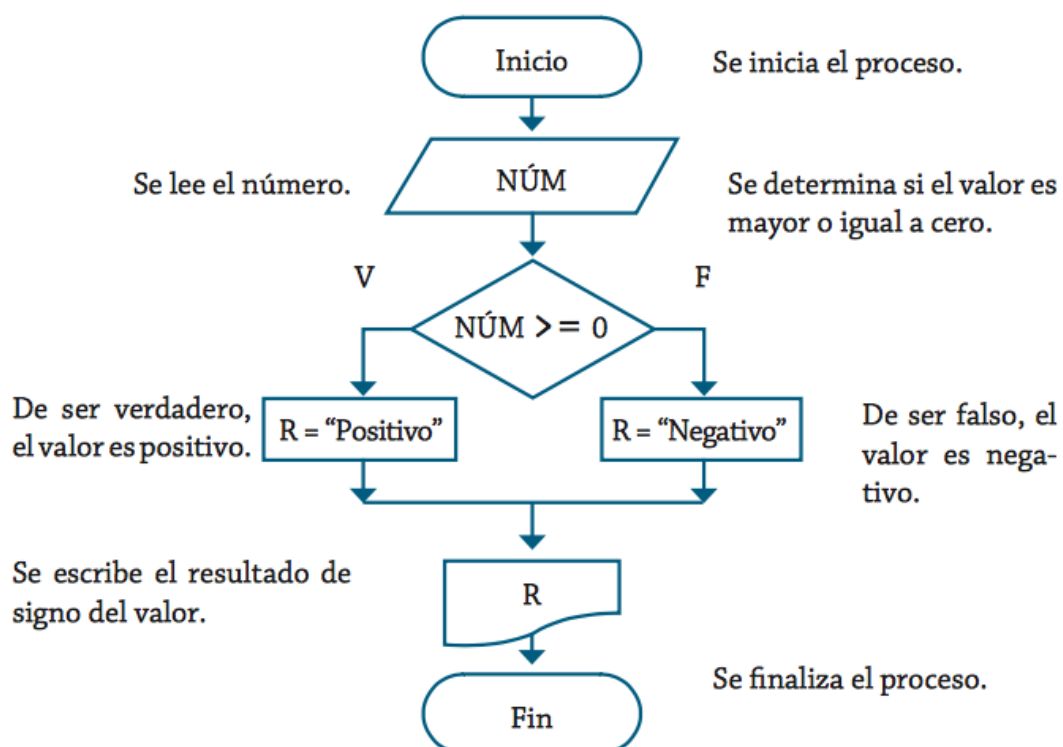


Símbolo	Significado
	Terminal /Inicio.
	Entrada de datos.
	Proceso.
	Decisión.



Ejemplo

Algoritmo que, a partir de 1 número, se indica si es positivo o negativo. Su representación gráfica mediante ordinograma podemos verla en el gráfico:



8.2 Pseudocódigo

Además de las representaciones gráficas, un programa puede describirse mediante un **lenguaje intermedio entre el lenguaje natural y el lenguaje de programación**, de tal manera que permita flexibilidad para expresar las acciones que se van a realizar y, también imponga algunas limitaciones, que tienen importancia cuando se quiere codificar el programa a un lenguaje de programación determinado.

La notación en pseudocódigo se caracteriza por:

- Facilitar la obtención de la solución mediante la utilización del diseño descendente o Top-down. Es una forma de codificar los programas o algoritmos fácil de aprender y utilizar.
- Posibilitar el diseño y desarrollar los algoritmos de una manera independiente del lenguaje de programación que se vaya a utilizar cuando se implemente el programa.
- Facilitar la traducción del algoritmo a un lenguaje de programación específico.
- Permitir un gran flexibilidad en el diseño del algoritmo que soluciona el problema, ya que se pueden representar las acciones de una manera más abstracta, no estando sometidas a las reglas tan rígidas que impone un lenguaje de programación.
- Posibilitar futuras correcciones y actualizaciones en el diseño del algoritmo por la utilización una serie de normas, que acotan el trabajo del desarrollador.

Cuando se escribe un algoritmo mediante la utilización de pseudocódigo, se debe **"sangrar"** el texto con respecto al margen izquierdo, con la finalidad de que se comprenda más fácilmente el diseño que se está realizando.

Ejemplo: algoritmo que lee dos números "X" e "Y", determina si son iguales, y en caso de no serlo, indica cuál de ellos es el mayor. Su representación gráfica mediante pseudocódigo será:

